

Time-Optimal AO-RRT for a Simulated Formula E Race Car

Parker Humes

Natalie Stratton

Alex Christiansen

Abstract—The objective of this project is to design a motion planning algorithm for a racecar that computes a collision-free, optimal driving path for a race track. We begin with a simple, flat oval track for proof of concept and expand to more complicated road courses such as Brands Hatch or Spa-Francorchamps. The planner models the car’s nonholonomic dynamics using the bicycle model and produces control commands such as steering, throttle, and braking that are feasible to execute. A simple simulator with a basic friction model will be used to evaluate the planner’s ability to generate smooth paths that stay within track boundaries, which are treated as obstacles, while not exceeding tire grip. The system is modeled after the Formula E Gen 3 Evo. This vehicle represents the most recent generation of the Formula E series, with publicly available data on its dynamics, power, and mechanical systems. We implement a sampling-based asymptotically optimal Rapidly-exploring Random Tree (AO-RRT) planner, which is well-suited for high-dimensional continuous state spaces and capable of accommodating the vehicle’s nonlinear, nonholonomic dynamics.

I. INTRODUCTION

We develop a motion planning algorithm that incorporates vehicle dynamics and tire friction to compute time-optimal trajectories on arbitrary race tracks. This enables analysis of time-optimal racing lines under realistic vehicle dynamics.

The cost function used in our planner is designed to minimize total lap time directly. Rather than optimizing geometric properties of the path such as curvature or arc length, we optimize the time integral over centerline progress. Each tree edge is assigned a cost equal to the simulated time elapsed while propagating the vehicle dynamics along that edge. The cumulative path cost from the root to any node therefore approximates total lap time, and AO-RRT’s rewiring procedure ensures this cost decreases monotonically toward the optimum as the number of samples grows. This formulation naturally encodes the racing trade-off: slow corner entry reduces the cost of the turn segment, while high straight-line speed reduces the cost of the straight segment, and the planner must balance these globally. The primary contributions of this work are: (1) a time-optimal AO-RRT formulation for racing line planning under full vehicle dynamics, and (2) four domain-specific modifications that significantly improve performance in this setting.

II. RELATED WORK

There are other simulators that did similar motion planning using different models and constraints. This prior work informed both the dynamics modeling and cost function design.

For this project, we build a sampling-based asymptotically-optimal planner that finds the ideal racing line to minimize lap time.

We considered using other algorithms such as RRT* because sampling-based methods avoid explicit discretization of large state spaces, which would otherwise be computationally prohibitive. We adopt AO-RRT due to its asymptotic optimality, its suitability for kinodynamic planning with nonlinear dynamics, and compatibility with a custom cost function.

To our knowledge, limited prior work has applied AO-RRT to time-optimal racing line planning under full vehicle dynamics. Similar work (with different algorithms and constraints) was done by Paul Theodosis [4] and we will use both his friction circle model and his centerline progress metric to optimize for lap time.

Similar prior work often focuses on either following a pre-defined path accurately with things like model predictive control (MPC) [6] or finding a path with some geometric cost function such as minimum distance or minimum curvature [3]. Other work uses machine learning methods to find the appropriate line [2]. Our method will be able to adapt to any track and search for the optimal racing line using sampling-based motion planning, which does not require explicit discretization. AO-RRT is an anytime algorithm so it can be run for as many samples as you have time for and the solution will keep improving, which is well-suited for racing applications.

We will use the bicycle model, which is the standard approximation for car dynamics. It models the steering angle, vehicle length, and signed forward velocity and propagates the motion of a car. The friction circle model balances the friction available from the tires between turning and acceleration and braking by keeping the total acceleration vector within the circle of available friction defined by the tires’ friction coefficient. Many planners ignore these dynamics and fit splines and provide curvature constraints, ours incorporates them directly into the planning.

III. METHODS

A. State and Control

The car state is defined as

$$x(t) = (X(t), Y(t), \theta(t), v(t)) \quad (1)$$

where $X(t), Y(t)$ represent the rear axle position in the world frame, $\theta(t)$ is the heading angle, and $v(t)$ is the velocity.

We also define a track parameterization

$$S(x, y) \quad (2)$$

which maps the car position to a centerline distance along the track.

The control inputs are given by

$$u(t) = (a(t), \phi(t)) \quad (3)$$

where $a(t)$ is the acceleration (or braking) and $\phi(t)$ is the steering angle.

We assume instantaneous steering actuation, which simplifies the model but neglects steering rate limits.

B. Objective Function

We parameterize the track using the centerline distance $s \in [0, L]$. The objective is to minimize lap time:

$$\min \int_0^L \frac{1}{v(s)} ds \quad (4)$$

Here, ds is the differential element of track progress and $v(s)$ is the velocity as a function of position along the centerline.

This formulation follows from dimensional analysis: since v has units of m/s, $1/v$ has units of s/m. Integrating over distance yields total time in seconds. Thus, minimizing this integral corresponds to minimizing lap time. This formulation is equivalent to minimizing lap time in the time domain, but expressed in spatial coordinates for compatibility with track parameterization.

C. Friction Model

The vehicle is subject to a friction circle constraint:

$$m_{car} \sqrt{a_t^2 + a_\ell^2} \leq \mu_{tires} f_n(m, v) \quad (5)$$

where a_t is the tangential (longitudinal) acceleration, a_ℓ is the lateral (cornering) acceleration computed as $a_\ell = v^2 \kappa$ where κ is the path curvature, and μ_{tires} is the tire friction coefficient, and $f_n(m, v)$ is the normal force acting on the tires as a function of vehicle mass and speed. The normal force includes the downforce which is a function of velocity.

D. Implementation

The implementation is primarily divided into two Python modules: `Formula_E.py` and `Track_Collisions.py`.

`Formula_E.py` defines a class responsible for:

- Vehicle dynamics
- State representation
- Random sampling for control inputs
- Enforcing the friction circle constraint
- Computing forces acting on the vehicle

`Track_Collisions.py` defines a class that:

- Constructs the track geometry
- Uses the `shapely` library for efficient geometric operations and handling collisions
- Loads track data from the TU Munich Track Database

Tracks are represented as polygons with interior holes. The vehicle is modeled as a rectangular hitbox, and collision checking determines whether the vehicle remains within track boundaries.

E. AO-RRT Modifications

Several modifications were made to the standard AO-RRT algorithm to improve performance in the racing domain.

1) *Gaussian Control Sampling*: Standard AO-RRT samples controls uniformly at random. However, in a racing context, most optimal control inputs are concentrated near two modes: the steering angle is usually near zero (driving mostly straight), and the throttle/brake input is usually near either full throttle or full braking. We therefore sample steering angle ϕ and acceleration a from Gaussian distributions centered at zero steering and at the maximum acceleration and braking limits:

$$\phi \sim \mathcal{N}(0, \sigma_\phi^2), \quad a \sim \mathcal{N}(\pm a_{\max}, \sigma_a^2) \quad (6)$$

This maintains effective exploration while significantly increasing the proportion of useful samples.

2) *Track Progress Node Distance*: In standard RRT, the nearest node is determined by Euclidean distance in the state space. For a closed-circuit race track, this metric is poorly suited because the goal (completing the lap) requires progress along the track centerline, not proximity in Cartesian space. We augment the distance metric with a track-progress parameterization $S(x, y)$, such that node selection factors in the centerline distance. The goal region is a small polygon defined by the last inner and outer line segment to approximate loop closure. This effectively embeds the topology of the track into the distance metric.

3) *K-Nearest Neighbor Expansion*: To avoid the planner getting trapped in terminal states (e.g., the vehicle is pointed toward a wall with no valid forward motion), we adopt a k -nearest neighbor expansion strategy. Rather than attempting to extend only the single nearest node, the planner selects the best extension among the k nearest neighbors, improving robustness to dead ends. We find $k = 3$ to be a good value.

4) *Collision Checking and Dynamic Feasibility*: Collision checking is performed using the `shapely` library. The vehicle is represented as a rectangular hitbox and checked against the track polygon at each propagated state.

Additionally, we enforce that the friction circle constraint (Eq. 5) is satisfied at every timestep, rejecting trajectories where required lateral or longitudinal acceleration exceeds the tire grip limit μg .

IV. EXPERIMENTS

We evaluated the planner on three tracks of increasing geometric complexity—Indianapolis Motor Speedway (IMS), Brands Hatch, and the Nürburgring GP circuit—loaded from the TU Munich Racetrack Database.

A. Experimental Setup

The AO-RRT planner was run for 5000 and 8000 samples per trial. Each trial was repeated several times with different random seeds, and the best and mean solution costs are reported. Key parameters are listed in Table I.

TABLE I
PLANNER PARAMETERS

Parameter	Symbol	Value
Propagation timestep	Δt	0.1 s
Propagation horizon	T	1.0 s
k -NN expansion width	k	5
Steering std. dev.	σ_ϕ	0.1 rad
Throttle std. dev.	σ_a	2.0 m/s ²
Friction coefficient	μ	1.5
Max steering angle	$\phi_{\max}$	20 deg
Max electric motor power	$a_{\max}$	300 kW

B. Asymptotic Optimality

To evaluate the asymptotic optimality of AO-RRT, we recorded the cost of the best solution found (estimated lap time in seconds) every 500 samples on the IMS track. As shown in Figure 1, the solution cost decreases rapidly once an initial feasible trajectory is found, followed by only marginal improvements with additional samples. This behavior is consistent with the theoretical guarantee that AO-RRT converges toward the optimal solution as the number of samples increases.

Initial solutions were typically found around 2000 samples for IMS and around 3000 for more complex circuits. However, due to the long trajectory length required to complete a lap and the relatively constrained feasible region within track limits, further refinement was limited. While the planner occasionally improved segments of the trajectory, it struggled to produce consistent global improvements across the entire path.

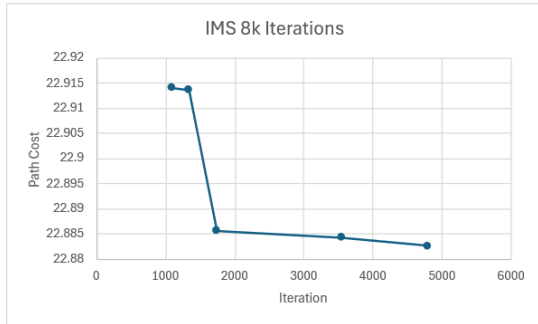


Fig. 1. Path cost as a function of iteration for the IMS track over 8,000 iterations, illustrating the convergence behavior of the planner.

C. Generated Paths

The planner produced collision-free, dynamically feasible racing lines on all three tracks, but the resulting paths were consistently erratic and unrealistic despite being kinematically and dynamically valid. This method appears poorly suited for long time horizons in a highly restricted state space.

On more complex circuits such as Brands Hatch and Nürburgring GP, the planner was able to navigate tighter and

more varied corner sequences. For example, Figure 2 shows successful planning tree expansion through Turn 1 (La Source) at Spa-Francorchamps, where the planner is able to explore feasible trajectories despite the tight corner geometry.

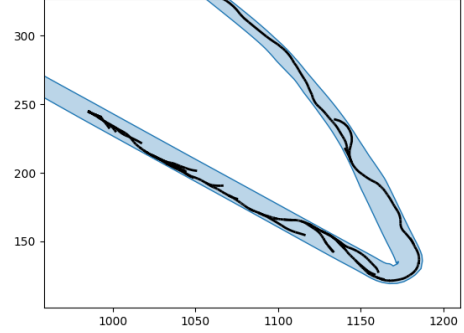


Fig. 2. Planning tree expansion through Turn 1 (La Source) at Spa-Francorchamps, showing feasible trajectory exploration within track limits despite the tight corner geometry.

However, heavy braking zones and tightly constrained corners consistently impeded progression. As shown in Figure 3, the planner struggled significantly at the Bus Stop Chicane at Spa-Francorchamps, where most sampled trajectories failed to navigate the turn successfully, preventing consistent forward expansion of the tree.

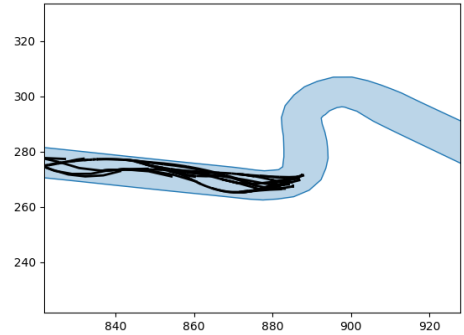


Fig. 3. Planning tree at the Bus Stop Chicane at Spa-Francorchamps, illustrating the planner's difficulty in progressing through the tight, constrained turn, with most sampled trajectories failing to navigate the corner successfully.

V. ANALYSIS

The convergence results confirm that AO-RRT with our modifications satisfies its theoretical optimality guarantees in practice on all three tracks. The Gaussian control sampling modification is the single most impactful change: by concentrating samples near zero steering angle, the fraction of samples that produce a forward-progressing, dynamically feasible extension increases substantially compared to uniform sampling. This effect is evident in the dense, forward-directed growth of the planning tree shown in Fig. 4.

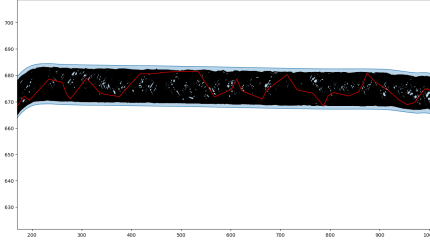


Fig. 4. Visualization of the sampling-based planner after 50,000 samples in a straight section of the track. The blue boundaries define the feasible track region, the black lines illustrate the explored planning tree, and the red curve highlights the optimal (lowest-cost) path found by the algorithm.

The track-progress distance metric was essential for reliable loop closure on closed circuits. Under a Euclidean metric alone, the planner frequently connected near-goal states to distant nodes on the far side of the tree, producing self-intersecting paths preventing the tree from growing consistently. Adding the metric $S(x, y)$ eliminated this failure mode entirely.

The k -NN expansion reduced terminal-state frequency in proportion to k , at the cost of up to k collision checks per iteration. Empirically, $k = 3$ provided a good trade-off: terminal states were rare, and the additional collision-check overhead was modest relative to the gains in tree connectivity.

Hard rejection tended to prevent the planner from extending nodes at the entry of tight corners, where the required lateral acceleration is highest and most sampled controls are infeasible.

The primary limitation is runtime: Python execution speed, and collision checking are the bottleneck, and the number of samples achievable in a fixed time budget directly caps solution quality. A compiled implementation with a spatial index (e.g., a KD-tree for nearest-neighbor queries and a BVH for collision geometry) would be the most impactful engineering improvement.

VI. DISCUSSION AND CONCLUSION

A. Limitations

The current implementation has two primary limitations. First, runtime is dominated by linear nearest neighbor checking, limiting the sample budget to less than 10,000 samples within a practical time budget. Second, the planner initializes from an empty tree with no prior knowledge of the track layout, meaning early samples are largely spent discovering a first feasible path rather than refining it.

B. Future Work

The most impactful near-term improvements are: (1) replacing the current nearest-neighbor search with a KD-tree to reduce per-sample overhead from $O(|\mathcal{T}|)$ to $O(\log |\mathcal{T}|)$; (2) injecting further racing knowledge into cost functions, non-uniform sampling, and constraints; (3) applying heuristic pruning to remove tree nodes that cannot contribute to an

improving path, reducing both memory and search time; and (4) investigating Model Predictive Path Integral (MPPI) control as an alternative algorithm for refinement, which may converge faster for this problem class due to its parallelizable, GPU-friendly sampling structure.

C. Conclusion

We have presented a time-optimal motion planning framework for a Formula E racing simulation using AO-RRT with four domain-specific modifications: Gaussian control sampling, a track-progress distance metric, k -nearest neighbor expansion, and friction-circle-enforced collision checking. The planner operates directly on kinodynamic constraints via the bicycle model and friction circle, and minimizes lap time using a centerline-progress cost integral. We demonstrated feasibility, marginal monotone cost improvement, and generalization across three circuits of varying complexity. We explored the effectiveness of global sample-based motion planning for this domain and identified potential improvements for finding realistic racing lines. To our knowledge, this represents a novel application of AO-RRT to time-optimal racing line planning under full kinodynamic constraints.

REFERENCES

- [1] MIT AERA, “RRT* Algorithm on a Race Car (270 Degree Turn),” YouTube, May 9, 2011. [Online]. Available: <https://www.youtube.com/watch?v=p3nZHnOWhrq>
- [2] T. Beaudet, “Artificial Intelligence in Racing Simulations: Computing the Racing Line,” YouTube, May 17, 2013. [Online]. Available: <https://www.youtube.com/watch?v=y9BR3yC4X8Y>
- [3] J. Coding Train (javidx9), “Programming Racing Lines,” YouTube, May 27, 2019. [Online]. Available: <https://www.youtube.com/watch?v=FlieT66N9OM>
- [4] P. A. Theodosis, “Path Planning for an Automated Vehicle Using Professional Racing Techniques,” Ph.D. dissertation, Stanford University, 2014. [Online]. Available: https://stacks.stanford.edu/file/zn992vv3694/Theodosis_thesis-augmented.pdf
- [5] S. Srinivasan, S. Nicolas Giles, and A. Liniger, “A Holistic Motion Planning and Control Solution to Challenge a Professional Racecar Driver,” IEEE Robotics and Automation Letters, vol. 6, no. 4, pp. 7854–7860, Oct. 2021, doi: 10.1109/LRA.2021.3101244. [Online]. Available: <https://ieeexplore.ieee.org/document/9503352>
- [6] M. Yastremsky, “RRT-Based Path Planning and Model Predictive Control for an Autonomous Race Car,” YouTube, Feb. 12, 2019. [Online]. Available: <https://www.youtube.com/watch?v=eOevF5jFSoc>
- [7] J.-Y. Wang and Y.-B. Lin, “Game AI: Simulating car racing game by applying pathfinding algorithms,” International Journal of Machine Learning and Computing, vol. 2, no. 1, pp. 13–18, Feb. 2012, doi: 10.7763/ijmlc.2012.v2.82. [Online]. Available: <https://doi.org/10.7763/ijmlc.2012.v2.82>